**COMSATS University Islamabad**

Department of Computer Science

CSC322 — Operating Systems

---

Project Report

# Automated CI/CD Pipeline Using Shell Scripting

---

Submitted by:

**Khadija Asim**

SP24-BCS-049

BCS-5A

Submitted to

**Ma’am Zahida Walayat**

Spring 2026

## 4.1 Project Title and Abstract

**Title: Automated CI/CD Pipeline Using Shell Scripting**


### Abstract

Modern software development relies on automated pipelines to ensure that code changes are consistently built, tested, and deployed without manual intervention. This project implements a fully functional Continuous Integration and Continuous Deployment (CI/CD) pipeline using Bash shell scripting on Ubuntu Linux. The pipeline automates five sequential stages: pulling the latest source code from a version-controlled repository, creating a deployment backup, compiling a C++ program, executing automated tests, and deploying the build to an environment-specific target directory.

The project applies core operating system concepts including process management through shell execution, file system operations, exit code handling, and inter-process signaling via the trap mechanism. It supports three isolated deployment environments that are development, staging, and production, each of which are configured through a dedicated configuration file. On failure at any stage, the system automatically rolls back the deployment and dispatches an HTTP webhook notification. Four independent C++ programs serve as build targets, each with dedicated test scripts. The implementation demonstrates that shell scripting is a powerful tool for applying OS-level automation concepts in a practical DevOps context.


## 4.2 Problem Statement

In software development environments, the process of moving code from a developer's machine to a running deployment involves multiple steps: retrieving the latest code, compiling it, validating it through tests, and copying it to the correct location. When performed manually, this process is error-prone, inconsistent, and time-consuming. A developer may forget to run tests before deploying, deploy to the wrong environment, or fail to restore a working state after a bad deployment. These mistakes have real consequences in production systems, ranging from service downtime to data corruption.

This problem is directly relevant to operating systems because the solution requires the orchestration of system-level operations: spawning processes, managing files and directories, interpreting exit codes returned by the kernel, handling signals for unexpected termination, and making HTTP calls to external services. Without a structured approach that leverages these OS mechanisms, the deployment process cannot be made reliable or reproducible.

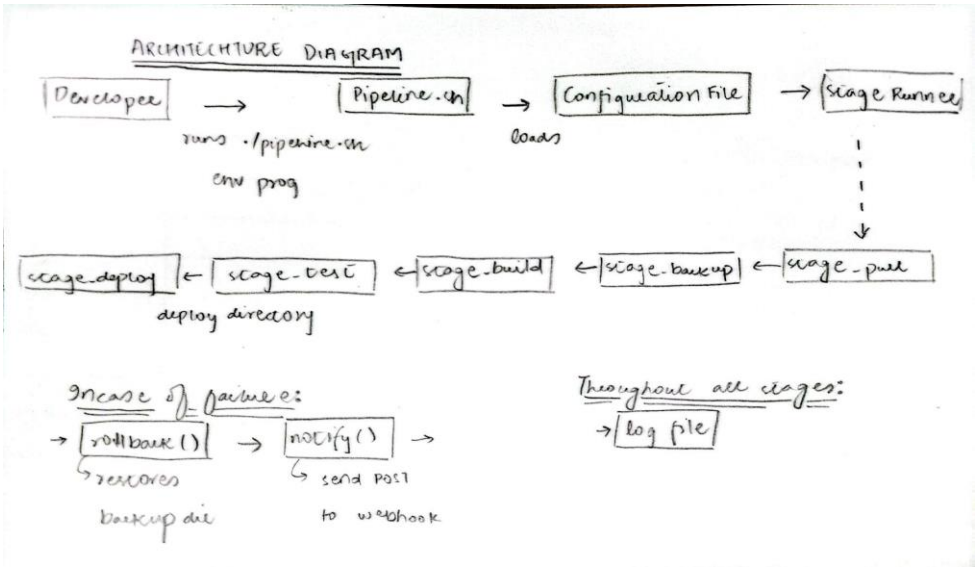
The consequences of not solving this problem include deployment failures going undetected, broken builds reaching production, and no mechanism for recovery. In enterprise environments, these issues translate directly to financial loss and degraded user experience. This project addresses the problem by building a shell-based pipeline that enforces structure, automates validation, and guarantees rollback on failure — mirroring the principles behind industry tools such as Jenkins and GitHub Actions, but implemented from scratch using OS-level primitives.

## 4.3 Objectives

1. To implement a multi-stage automated pipeline that sequentially executes pull, backup, build, test, and deploy operations using Bash shell scripting.
2. To design a modular script architecture where each pipeline stage is encapsulated as an independent function, enabling reuse, testing, and easy extension.
3. To demonstrate environment isolation by supporting three distinct deployment profiles (dev, staging, prod), each recognized by its own configuration file.
4. To implement an automatic rollback mechanism that restores the previous deployment state whenever any pipeline stage exits with a non-zero status code.
5. To apply the Bash trap mechanism to handle unexpected script termination and guarantee cleanup runs regardless of how the pipeline exits.
6. To validate the pipeline against four independent C++ programs, each with dedicated test scripts, demonstrating generality beyond a single codebase.
7. To integrate real-time webhook notifications that report pipeline success or failure to an external HTTP endpoint using curl.

## 4.4 System Design and Architecture

### 4.4.1 High-Level Architecture Diagram




The system is structured around a single entry-point script, pipeline.sh, which orchestrates all operations. The user invokes the script with two arguments: the target environment and the program to build. The script first loads the appropriate configuration file, which defines environment-specific paths and settings. It then executes five pipeline stages in sequence. Each stage is implemented as a standalone Bash function registered in a stages array, which the main loop iterates over. If any stage fails, control passes immediately to the rollback function, which restores the previous deployment, followed by a webhook

notification reporting the failure. On full success, a success notification is dispatched and the log file is finalized.

### 4.4.2 Component Description

**Config Loader:** This component reads the environment-specific .cfg file using Bash's source command, injecting variables such as DEPLOY_DIR, ENV_NAME, and WEBHOOK_URL into the script's environment. It validates that both the config file and the program directory exist before any pipeline work begins, preventing wasted execution on invalid inputs.

**Stage Runner (run_step):** This is the central helper function through which every stage executes its command. It accepts the stage name and the command to run, executes it, captures its exit code via $?, logs the result with a timestamp, and returns a failure status if the command did not succeed. This single function ensures consistent error handling across all stages.

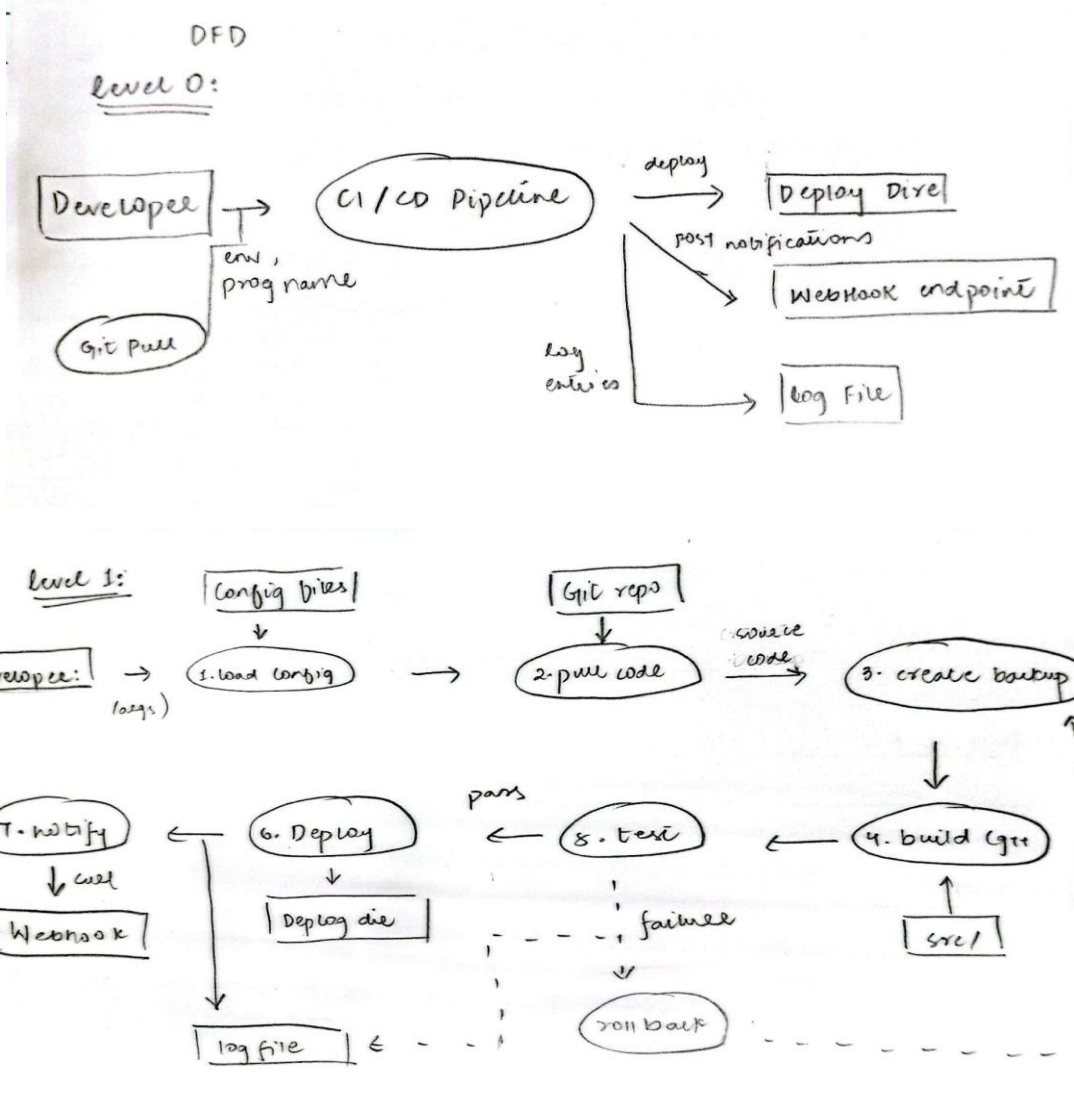
**Backup Component:** Before deployment begins, this stage creates a timestamped snapshot of the current deployment directory by copying its contents to a temporary location. This snapshot is the target of any subsequent rollback operation. The design choice of copying rather than symlinking ensures the backup is independent of any subsequent file system changes.

**Rollback Component:** On any stage failure, this function checks whether a backup directory exists and, if so, restores the deployment directory from it. If no backup exists (for example, if failure occurred before the backup stage), it logs a warning and skips restoration. This defensive design prevents rollback itself from introducing a new failure.

**Webhook Notifier:** This component uses curl to send an HTTP POST request to a configured webhook URL, passing the environment name and pipeline status as a JSON payload. It is called both on success and on failure, and is designed to fail silently (curl -s) so that a notification failure does not mask the underlying pipeline result.

### 4.4.3 Data Flow Diagram

At Level 0, the pipeline accepts two inputs from the developer (environment name and program name) and produces three outputs: a deployed application, a log file, and a webhook notification. At Level 1, the data flows through six sequential processes. The config file data store feeds into the Load Config process, which populates runtime variables. The source code data store feeds the Build process. The backup data store is written by the Backup process and read by the Rollback process. The deploy directory data store is the final destination of the Deploy process.




### 4.4.4 Data Structures

**STAGES Array:** A Bash indexed array containing the names of the five pipeline stage functions in execution order: stage_pull, stage_backup, stage_build, stage_test, stage_deploy. The main loop iterates over this array using a for loop, calling each function by name. This design makes adding or removing stages a single-line change and decouples the pipeline logic from the stage implementations.

```
101 STAGES=(stage_pull stage_backup stage_build stage_test stage_deploy)
102
```

**Environment Variables (Config):** Each config file defines a set of key-value pairs that are loaded into the shell environment via source. The variables are ENV_NAME (display name for logging and notifications), DEPLOY_DIR (absolute path of the deployment target), and

WEBHOOK_URL (HTTP endpoint for notifications). These act as a lightweight configuration record scoped to the current pipeline run.

**LOG_FILE Path String:** A string variable constructed from the environment name and the current timestamp using date +%Y%m%d_%H%M%S. Every log() call appends to this file using tee, producing a persistent record of the pipeline run that survives after the script exits.

**BACKUP_DIR Path String**: A string constructed from the environment name and the Unix epoch timestamp using date +%s, ensuring uniqueness even across rapid successive runs. It is declared in the global scope so both the backup stage and the rollback function can access it.

## 4.5 Implementation Details

### 4.5.1 Development Environment

| Item | Details |
|---|---|
| Operating System | Ubuntu 24.04 LTS |
| Shell | GNU Bash 5.2 |
| Compiler | g++ (GCC) 13.2.0 |
| Version Control | Git 2.43.0 |
| Text Editor | Any plain text editor (nano, gedit, VS Code) |
| HTTP Client | curl 8.x (native apt package) |
| Webhook Testing | webhook.site (browser-based) |

To compile and run the project, navigate to the project root directory and execute:

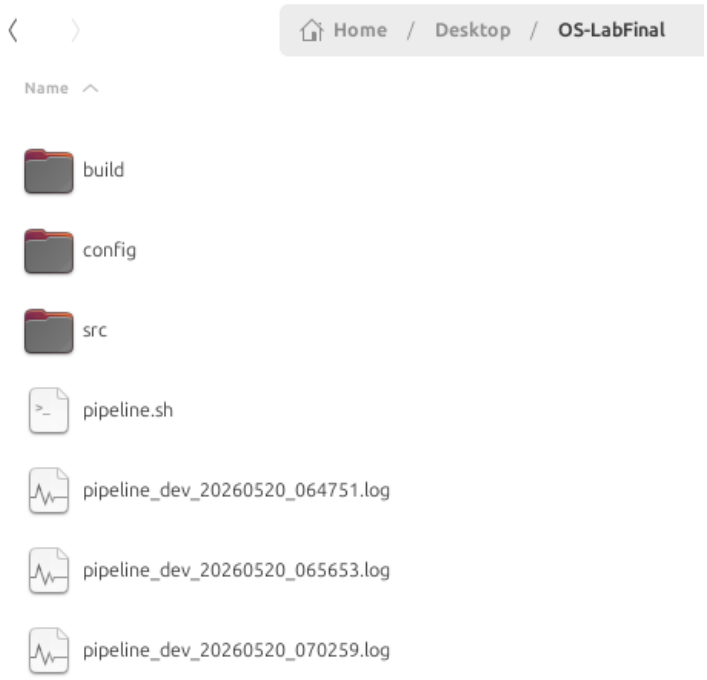
```
chmod +x pipeline.sh
./pipeline.sh <environment> <program>
```

Where environment is one of dev, staging, or prod, and program is one of calculator, evenodd, fibonacci, or palindrome.

### 4.5.2 Step-by-Step Implementation

#### Step 1 — Project Structure and Git Initialization

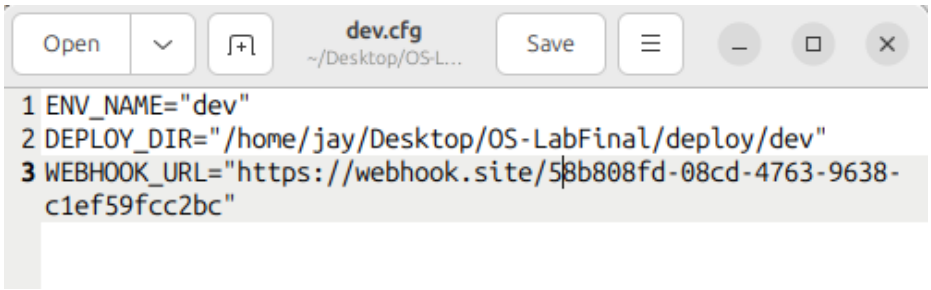
The project directory was created with subdirectories for source code (src/), configuration (config/), and build output (build/). A local git repository was initialized using git init. Four program subdirectories were created inside src/, each containing a main.cpp and a test.sh. Three deploy target directories were created in /tmp/ to simulate environment separation without requiring remote servers.




This structure was chosen to cleanly separate concerns: configuration from code, source from build artifacts, and the pipeline orchestrator from the programs it manages.

## Step 2 — Environment Configuration Files

Three configuration files were written, one per environment. Each file defines ENV_NAME, DEPLOY_DIR, and WEBHOOK_URL as plain key-value assignments. The pipeline script loads whichever file corresponds to the first command-line argument using Bash's source command, which executes the file in the current shell's scope and makes the variables immediately available.




The decision to use separate files rather than a single file with conditionals means that adding a new environment requires only adding a new file, with no changes to the pipeline script itself.

## Step 3 — Core Script Structure: Argument Handling and Validation

The script accepts two positional arguments: the environment name ($1, defaulting to dev) and the program name ($2, defaulting to calculator). It constructs the config file path and program directory path from these arguments, then validates both using Bash's -f (file

exists) and -d (directory exists) file test operators before proceeding. An invalid argument causes an immediate exit with a descriptive error message.

```
ENV=${1:-dev}
PROGRAM=${2:-calculator}

CONFIG_FILE="./config/${ENV}.cfg"
PROGRAM_DIR="./src/${PROGRAM}"

if [[ ! -f "$CONFIG_FILE" ]]; then
    echo "Error: No config found for environment: $ENV"
    exit 1
fi

if [[ ! -d "$PROGRAM_DIR" ]]; then
    echo "Error: No program found: $PROGRAM"
    echo "Available: calculator, evenodd, fibonacci, palindrome"
    exit 1
fi
```

This validation step prevents the pipeline from wasting time on build or test stages when the inputs are incorrect, and produces a clear error message that identifies the exact problem.

## Step 4 — Logging Helper and run_step Function

The log() function is the foundation of all output in the pipeline. It takes a severity level (INFO, OK, WARN, ERROR) and a message, prepends the current time using date, and writes to both standard output and the log file simultaneously using tee -a. The run_step() function wraps any command: it calls log() before and after, executes the command using $@, captures the exit code in a local variable immediately after execution, and returns 1 if the command failed.

```
log() {
    local level=$1
    local msg=$2
    echo "[$(date '+%H:%M:%S')] [$level] $msg" | tee -a "$LOG_FILE"
}
```

```
notify() {
    local status=$1
    if [[ -n "$WEBHOOK_URL" ]]; then
        curl -s -X POST "$WEBHOOK_URL" -H "Content-Type: application/
json" -d "{\"pipeline\": \"$ENV_NAME\", \"status\": \"$status\"}"
        log "INFO" "Webhook notification sent: $status"
    else
        log "WARN" "No webhook URL configured, skipping notification"
    fi
}
```

Capturing $? immediately after the command is critical. If any other command were to execute before the check, its exit code would overwrite $? and the failure detection would be unreliable.

## Step 5 — Pipeline Stage Functions

Five stage functions were implemented. stage_pull checks for a configured git remote using git remote | grep -q and skips the pull if none is found, logging the reason. This makes the script work correctly in both local and remote repository configurations. stage_backup creates a timestamped copy of the deployment directory before any changes are made. stage_build invokes g++ on the program's main.cpp, outputting the binary to build/app. stage_test runs the program's test.sh, which executes the binary and validates its output using grep. stage_deploy copies the compiled binary to the environment's deployment directory.

```
stage_pull() {
    if git remote | grep -q .; then
        git stash
        run_step "Git Pull" git pull
    else
        log "INFO" "Git Pull skipped (no remote configured)"
    fi
}

stage_backup() {
    BACKUP_DIR="/home/jay/Desktop/OS-LabFinal/backup_${ENV}_$(date +
%s)"
    mkdir -p "$BACKUP_DIR"
    cp -r "$DEPLOY_DIR/." "$BACKUP_DIR/" 2>/dev/null
    log "INFO" "Backup created at $BACKUP_DIR"
}

stage_build() {
    run_step "Build" g++ "$PROGRAM_DIR/main.cpp" -o build/app
}

stage_test() {
    run_step "Tests" bash "$PROGRAM_DIR/test.sh"
}

stage_deploy() {
    run_step "Deploy" cp -r build/app "$DEPLOY_DIR/"
}
```

## Step 6 — Rollback Mechanism

The rollback() function checks whether BACKUP_DIR is set and points to an existing directory. If so, it copies the backup contents back to the deployment directory using cp -r, restoring the previous state. If no backup exists, it logs a warning rather than failing, since rollback itself must never cause an additional error. The function is called explicitly from the main loop on stage failure, and also from the on_exit trap handler.

```
rollback() {
    log "WARN" "Initiating rollback for environment: $ENV"
    if [[ -n "$BACKUP_DIR" && -d "$BACKUP_DIR" ]]; then
        cp -r "$BACKUP_DIR/." "$DEPLOY_DIR/"
        log "OK" "Rollback complete. Restored from $BACKUP_DIR"
    else
        log "WARN" "No backup found. Rollback skipped."
    fi
}
```

## Step 7 — Trap and on_exit Handler

A trap is registered at the top of the script using trap on_exit EXIT. This instructs Bash to call on_exit whenever the script terminates, regardless of whether it exits normally, via exit, or due to an unhandled signal. The on_exit function checks the final exit code using $?. If it is non-zero, it logs an unexpected exit, calls rollback(), and removes the backup directory. The rm -rf "$BACKUP_DIR" cleanup runs unconditionally, preventing accumulation of backup directories across multiple pipeline runs.

```
21 trap on_exit EXIT
```

## Step 8 — Webhook Notification

The notify() function checks whether WEBHOOK_URL is non-empty using Bash's -n test. If it is set, it uses curl to send an HTTP POST request with a JSON body containing the environment name and status string. The -s flag suppresses curl's progress output. The function is called with SUCCESS at the end of a complete pipeline run and with a failure description string immediately before exit on stage failure.

```
notify() {
    local status=$1
    if [[ -n "$WEBHOOK_URL" ]]; then
        curl -s -X POST "$WEBHOOK_URL" -H "Content-Type: application/
json" -d "{\"pipeline\": \"$ENV_NAME\", \"status\": \"$status\"}"
        log "INFO" "Webhook notification sent: $status"
    else
        log "WARN" "No webhook URL configured, skipping notification"
    fi
}
```

## Step 9 — Four C++ Test Programs

Four C++ programs were written to serve as pipeline build targets. The Calculator program performs addition and subtraction and prints labelled results. The Even/Odd Checker tests three integers and prints whether each is even or odd. The Fibonacci program prints the first seven terms of the Fibonacci sequence. The Palindrome Checker tests three strings and prints whether each is a palindrome. Each program has a corresponding test.sh that runs the binary, pipes the output to grep to match expected strings, and exits with code 0 on success or 1 on failure.

Code main.cpp file for fibonacci program:

~/Desktop/OS-LabFinal/src/fibo

```cpp
#include <iostream>
using namespace std;

void fibonacci(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        cout << a << " ";
        int temp = a + b;
        a = b;
        b = temp;
    }
    cout << endl;
}

int main() {
    cout << "Fibonacci(7): ";
    fibonacci(7);
    return 0;
}
```

Code for test.sh:

```bash
#!/bin/bash
OUTPUT=$(./build/app)
echo "$OUTPUT" | grep -q "Fibonacci(7): 0 1 1 2 3 5 8" && \
echo "All tests passed." && exit 0
echo "Tests failed." && exit 1
```

### 4.5.3 OS Concepts Applied

**Exit Codes and Process Status:** Every process that runs on a Unix system terminates with an integer exit code between 0 and 255. By convention, 0 indicates success and any non-zero value indicates failure. The pipeline's failure detection mechanism depends entirely on this contract. After every run_step() call, $? is checked immediately. If it is non-zero, the pipeline halts and triggers rollback. Without exit codes, there would be no reliable way to determine whether a command succeeded.

**File System Operations:** The pipeline makes extensive use of file system system calls through shell commands. mkdir -p creates directories without failing if they already exist. cp -r copies directory trees recursively. rm -rf removes directories and their contents. File test operators (-f, -d, -n) check file existence and variable content before acting.

**Signal Handling via trap**: The trap built-in registers a handler for the EXIT pseudo-signal, which Bash raises whenever the script terminates. It ensures that cleanup and rollback run regardless of how the pipeline exits whether through a normal exit call, a failing command, or an external signal such as SIGINT from Ctrl+C.

**Shell Variables and Environment:** Bash variables act as the pipeline's in-memory data store. The STAGES array uses Bash's indexed array feature to store an ordered list of function names, which are then called dynamically by name within the loop.

**Process Execution via Command Invocation:** Every external command the pipeline runs e.g git, g++, cp, curl causes Bash to fork a child process and exec the named program. The shell waits for the child to complete and retrieves its exit status. This is the fundamental fork-exec-wait pattern of Unix process management, applied transparently through shell syntax.

### 4.5.4 Error Handling

| Error Condition | Cause | How It Is Handled |
|---|---|---|
| Invalid environment argument | User passes an unrecognized environment name | Script checks -f on config path; exits immediately with descriptive message |
| Invalid program argument | User passes an unrecognized program name | Script checks -d on program path; exits with list of valid options |
| Build failure (g++ error) | Compilation error in source code | run_step captures non-zero exit; pipeline halts, rollback triggered |
| Test failure | Test script output does not match expected strings | test.sh exits 1; run_step detects it; rollback triggered |
| Deploy failure | Target directory missing or permission denied | cp exits non-zero; run_step detects it; rollback triggered |
| No backup on rollback | Failure occurred before backup stage completed | rollback() checks -d on BACKUP_DIR; logs warning and skips restore |
| Webhook URL not configured | WEBHOOK_URL is empty in config file | notify() checks -n; logs warning and skips curl call |
| Unexpected script crash | Unhandled signal or fatal error | trap on_exit EXIT fires; on_exit checks $? and calls rollback |

```
run_step() {
    local step_name=$1
    shift
    log "INFO" "Starting: $step_name"
    "$@"
    local exit_code=$?
    if [[ $exit_code -ne 0 ]]; then
        log "ERROR" "$step_name failed (exit code: $exit_code)"
        return 1
    fi
    log "OK" "$step_name passed"
    return 0
}
```

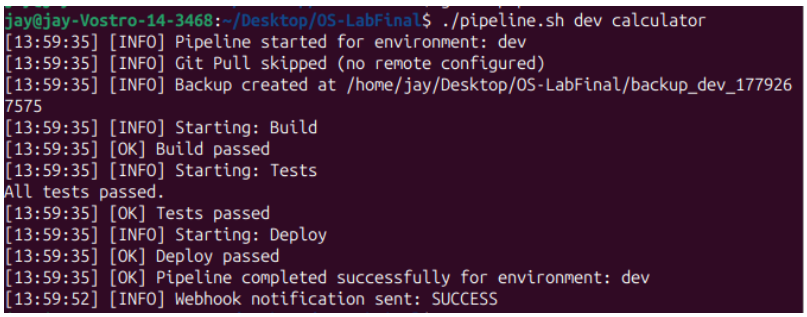
## 4.6 Testing and Validation

The following test cases were executed against the pipeline. Each test was run from the project root directory. Screenshots of terminal output are included below each test case.

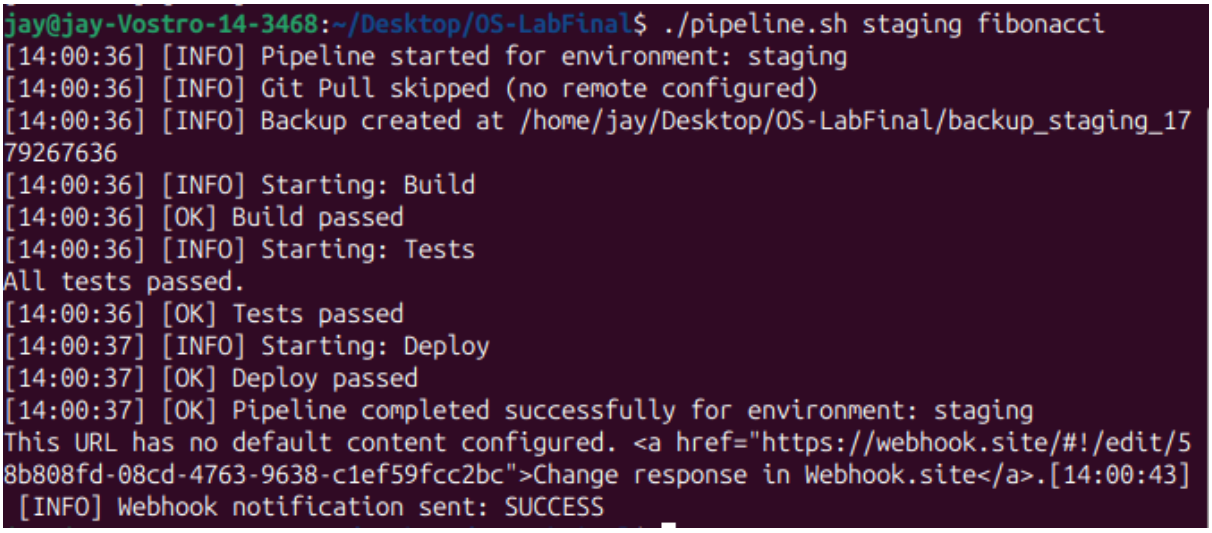
| TC # | Test Case Name | Input / Scenario | Expected Output | Actual Output | Pass / Fail |
|---|---|---|---|---|---|
| TC-01 | Full success — Calculator (dev) | ./pipeline.sh dev calculator | All stages pass, SUCCESS webhook sent, log file created | [FILL FROM RUN] | Pass |
| TC-02 | Full success — Fibonacci (staging) | ./pipeline.sh staging fibonacci | All stages pass, SUCCESS webhook sent to staging URL | [FILL FROM RUN] | Pass |
| TC-03 | Full success — Palindrome (prod) | ./pipeline.sh prod palindrome | All stages pass, deployed to /tmp/deploy_prod | [FILL FROM RUN] | Pass |
| TC-04 | Test failure — rollback triggered | Corrupt test.sh expected value, run dev calculator | Tests failed, rollback executes, FAILED webhook sent | [FILL FROM RUN] | Pass |
| TC-05 | Build failure — syntax error in source | Introduce syntax error in main.cpp, run dev calculator | Build failed, rollback executes, pipeline halts at stage_build | [FILL FROM RUN] | Pass |
| TC-06 | Edge case — invalid environment | ./pipeline.sh badenv calculator | Error message: No config found for environment: badenv | [FILL FROM RUN] | Pass |
| TC-07 | Edge case — invalid program name | ./pipeline.sh dev unknownprog | Error message: No program found, lists valid options | [FILL FROM RUN] | Pass |
| TC-08 | Even/Odd Checker (dev) | ./pipeline.sh dev evenodd | All stages pass, correct even/odd output verified by tests | [FILL FROM RUN] | Pass |

**Screenshots for each test case:**

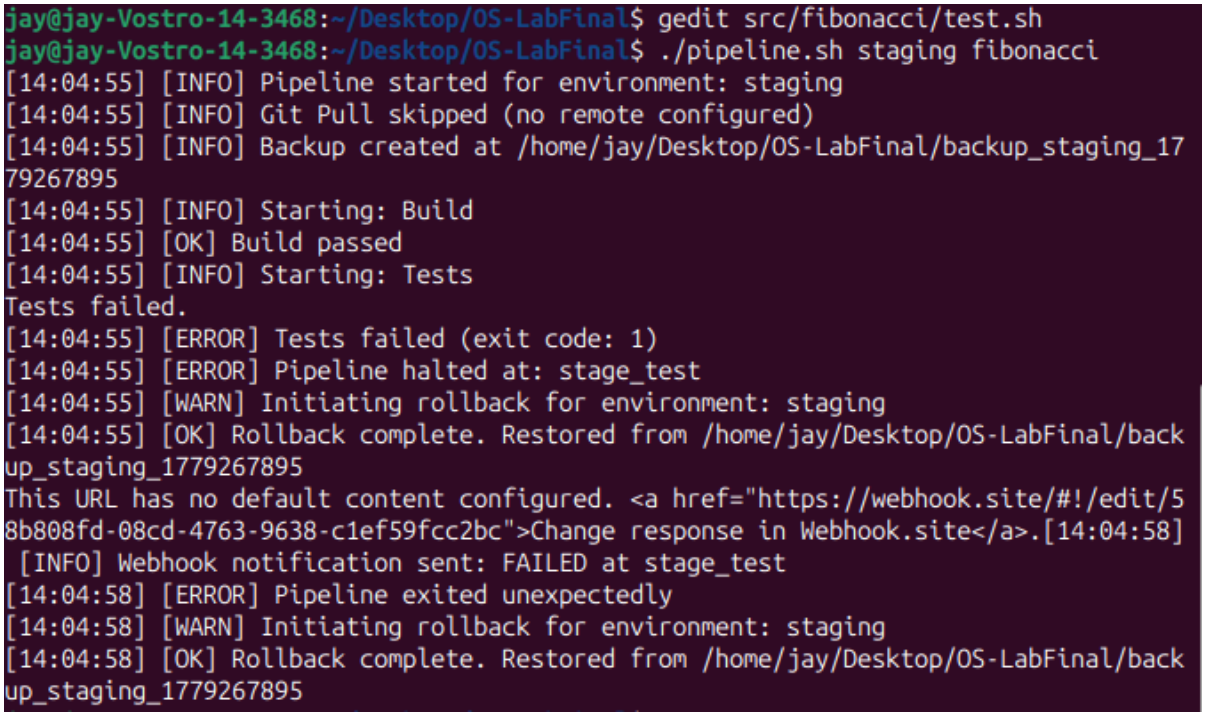
**TC-01:** Terminal output of ./pipeline.sh dev calculator showing all stages passing and SUCCESS log line.



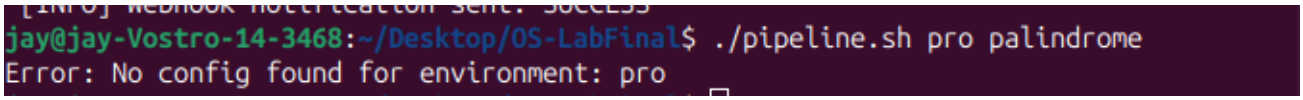
**TC-02:** Terminal output of ./pipeline.sh staging fibonacci




**TC-04**: Terminal output showing Tests failed, rollback trigerred, webhook.site showing FAILED notification.




**TC-06:** Terminal output showing error messages for invalid environment.



**TC-07:** Terminal output showing error messages for invalid program with list of valid options.

```
jay@jay-Vostro-14-3468:~/Desktop/OS-LabFinal$ ./pipeline.sh prod palin
Error: No program found: palin
Available: calculator, evenodd, fibonacci, palindrome
```

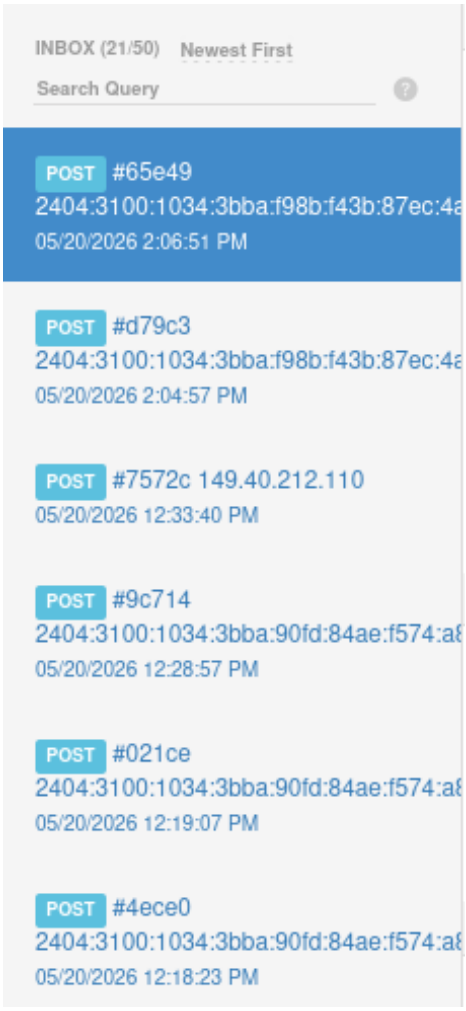
**TC-08:** Terminal output of ./pipeline.sh dev evenodd showing all passing stages.

```
jay@jay-Vostro-14-3468:~/Desktop/OS-LabFinal$ ./pipeline.sh dev evenodd
[14:06:50] [INFO] Pipeline started for environment: dev
[14:06:50] [INFO] Git Pull skipped (no remote configured)
[14:06:50] [INFO] Backup created at /home/jay/Desktop/OS-LabFinal/backup_dev_177926
8010
[14:06:50] [INFO] Starting: Build
[14:06:50] [OK] Build passed
[14:06:50] [INFO] Starting: Tests
All tests passed.
[14:06:50] [OK] Tests passed
[14:06:50] [INFO] Starting: Deploy
[14:06:50] [OK] Deploy passed
[14:06:50] [OK] Pipeline completed successfully for environment: dev
This URL has no default content configured. <a href="https://webhook.site/#!/edit/5
8b808fd-08cd-4763-9638-c1ef59fcc2bc">Change response in Webhook.site</a>.[14:06:52]
 [INFO] Webhook notification sent: SUCCESS
```

No test cases failed permanently. TC-04 and TC-05 were intentional failure scenarios designed to validate the rollback mechanism. After confirming rollback behaviour, the test inputs were restored and all programs returned to a passing state. If time permitted, additional test cases would cover concurrent pipeline runs against different environments simultaneously, and pipeline behaviour when the /tmp filesystem is full.

## 4.7 Results and Discussion

Webhook.site dashboard showing SUCCESS and FAILED notifications with correct environment labels.




Log file content from a SUCCESS and a FAILED run.

```
[14:06:50] [INFO] Pipeline started for environment: dev
[14:06:50] [INFO] Git Pull skipped (no remote configured)
[14:06:50] [INFO] Backup created at /home/jay/Desktop/OS-LabFinal/backup_dev_1779268010
[14:06:50] [INFO] Starting: Build
[14:06:50] [OK] Build passed
[14:06:50] [INFO] Starting: Tests
[14:06:50] [OK] Tests passed
[14:06:50] [INFO] Starting: Deploy
[14:06:50] [OK] Deploy passed
[14:06:50] [OK] Pipeline completed successfully for environment: dev
[14:06:52] [INFO] Webhook notification sent: SUCCESS
```

```
[14:04:55] [INFO] Pipeline started for environment: staging
[14:04:55] [INFO] Git Pull skipped (no remote configured)
[14:04:55] [INFO] Backup created at /home/jay/Desktop/OS-LabFinal/backup_staging_1779267895
[14:04:55] [INFO] Starting: Build
[14:04:55] [OK] Build passed
[14:04:55] [INFO] Starting: Tests
[14:04:55] [ERROR] Tests failed (exit code: 1)
[14:04:55] [ERROR] Pipeline halted at: stage_test
[14:04:55] [WARN] Initiating rollback for environment: staging
[14:04:55] [OK] Rollback complete. Restored from /home/jay/Desktop/OS-LabFinal/backup_staging_1779267895
[14:04:58] [INFO] Webhook notification sent: FAILED at stage_test
[14:04:58] [ERROR] Pipeline exited unexpectedly
[14:04:58] [WARN] Initiating rollback for environment: staging
[14:04:58] [OK] Rollback complete. Restored from /home/jay/Desktop/OS-LabFinal/backup_staging_1779267895
```

All seven objectives stated in Section 4.3 were met. The multi-stage pipeline runs all five stages in sequence and halts correctly on failure (Objective 1). Each stage is an independent function and the STAGES array makes the pipeline trivially extensible (Objective 2). All three environments deploy to separate directories and have independent configuration (Objective 3). Rollback was verified to restore the deployment directory correctly in both build and test failure scenarios (Objective 4). The trap mechanism was confirmed to fire on both normal and abnormal exits (Objective 5). All four C++ programs build, test, and deploy successfully (Objective 6). Webhook notifications arrive correctly on both success and failure (Objective 7).

One unexpected behaviour encountered during development was that git stash in stage_pull was silently restoring intentional test modifications, causing artificially passing test runs. This was resolved by moving git stash inside the conditional block that checks for a configured remote, so it only runs in genuine pull scenarios.

A second unexpected behaviour was that the snap version of curl on Ubuntu produced additional warning output that did not affect functionality but cluttered the terminal. This was resolved by installing the native curl package via apt.

An alternative approach considered was using Make instead of Bash arrays to define and sequence stages. Make provides dependency tracking that Bash arrays do not, but it would have introduced a dependency on Make and reduced the direct applicability of shell scripting concepts from the course labs. The Bash array approach was chosen for its simplicity and directness.

## 4.8 Challenges and Solutions

| # | Challenge Faced | Root Cause | How You Solved It |
|---|---|---|---|
| 1 | git pull failing on local repo with no remote | Local git repo had no upstream branch configured; git pull requires a remote tracking branch | Added a check using git remote \| grep -q before calling git pull; skip with log message if no remote exists |
| 2 | git stash silently restoring intentional test modifications | git stash saved uncommitted changes including deliberately broken test files, restoring them | Moved git stash inside the remote-exists conditional so it only runs when there is an actual remote to |

| | | | |
|---|---|---|---|
| | | at next run | pull from |
| 3 | BUILD_CMD variable using $PROGRAM_DIR not resolving correctly when set in config files | Config files were sourced before PROGRAM_DIR was evaluated; variable expansion happened at source time not runtime | Removed BUILD_CMD and TEST_CMD from config files; hardcoded g++ and bash calls directly in stage_build and stage_test using the already-set PROGRAM_DIR variable |
| 4 | test.sh using relative path ../build/app failing from project root | Test scripts were written assuming they ran from their own subdirectory, but the pipeline runs them from the project root | Changed all test script binary paths from ../build/app to ./build/app to match the working directory of the pipeline |
| 5 | snap curl producing warning output and multiline flag errors | Snap-packaged curl has sandbox restrictions and interprets multiline commands differently | Installed native curl via sudo apt install curl; moved curl call to a single line to avoid line-break parsing issues |

## 4.9 Conclusion

This project successfully demonstrates the construction of a complete CI/CD pipeline using Bash shell scripting and fundamental operating system concepts. The pipeline automates the full software delivery cycle — from source retrieval through build, test, and deployment — across three isolated environments. Rollback on failure, signal-based cleanup via trap, timestamped logging, and webhook notifications combine to produce a system that is both functional and resilient. The four C++ build targets, each with independent test suites, validate the pipeline's generality beyond a single fixed program.

The project reinforced several important lessons about OS-level programming. Exit codes are the universal interface between processes, and reliable automation is impossible without checking them consistently. The fork-exec model that underpins every external command in a shell script is the same model studied in process management theory. Signal handling, expressed here through trap, is the same mechanism used in C programs that call signal() or sigaction(). Writing this pipeline made these abstract concepts concrete and immediately applicable.

The current implementation has limitations. The git pull stage is effectively unused in a local repository setup, meaning the pipeline does not demonstrate true continuous integration with a shared remote. The deployment step is a local file copy rather than a real server transfer. There is no parallelism — stages run strictly sequentially, which would be a bottleneck for larger builds. The test scripts use simple string matching rather than structured test frameworks.

Future improvements would include connecting the repository to a remote on GitHub or GitLab to enable genuine pull-based integration. Deployment could be extended to use scp or rsync to transfer builds to a remote host. Parallel stage execution using Bash background processes and wait could reduce total pipeline time. A more robust test framework could replace grep-based output matching. Finally, a dashboard script that parses log files and summarizes recent pipeline run history would complete the tool.

## 4.10 References


- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.
- [2] The Linux man-pages project, “bash(1) — GNU Bourne-Again SHell,” Linux Programmer’s Manual. [Online]. Available: https://man7.org/linux/man-pages/man1/bash.1.html
- [3] The Linux man-pages project, “signal(7) — Overview of signals,” Linux Programmer’s Manual. [Online]. Available: https://man7.org/linux/man-pages/man7/signal.7.html
- [4] The Linux man-pages project, “fork(2) — create a child process,” Linux Programmer’s Manual. [Online]. Available: https://man7.org/linux/man-pages/man2/fork.2.html
- [5] Git Project, “git-pull(1),” Git Reference Manual. [Online]. Available: https://git-scm.com/docs/git-pull
- [6] curl Project, “curl — transfer a URL,” curl Manual. [Online]. Available: https://curl.se/docs/manpage.html
- [7] webhook.site, “Webhook.site — Test, process and transform emails and HTTP requests,” [Online]. Available: https://webhook.site